

CHAPTER 7

Lists and Tuples

starting out with >>>

PYTHON®

FOURTH EDITION



TONY GADDIS

Topics

- Sequences 
- Introduction to Lists 
- List Slicing 
- Finding Items in Lists with the in Operator 
- List Methods and Useful Built-in Functions 

Topics (cont'd.)

- Copying Lists 
- Processing Lists 
- Two-Dimensional Lists 
- Tuples 
- Plotting List Data with the `matplotlib` Package 

Topics

- **Sequences**
- Introduction to Lists
- List Slicing
- Finding Items in Lists with the in Operator
- List Methods and Useful Built-in Functions

Sequences

- **Sequence: an object that contains multiple items of data**
 - The items are stored in sequence one after another
- **Python provides different types of sequences, including lists and tuples**
 - The difference between these is that a list is mutable and a tuple is immutable



Topics

- Sequences
- **Introduction to Lists**
- List Slicing
- Finding Items in Lists with the in Operator
- List Methods and Useful Built-in Functions

Introduction to Lists

- **List**: an object that contains multiple data items
 - **Element**: An item in a list
 - Format: `list = [item1, item2, etc.]`
 - Can hold items of different types
- **print function** can be used to display an entire list
- **list()** function can convert certain types of objects to lists

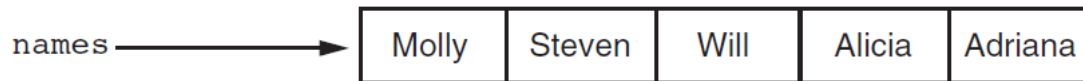
Introduction to Lists (cont'd.)

Figure 7-1 A list of integers



```
even_numbers = [2, 4, 6, 8, 10]    Print(even_numbers)
```

Figure 7-2 A list of strings



```
names = ['Molly', 'Steven', 'Will', 'Alicia', 'Adriana']
```

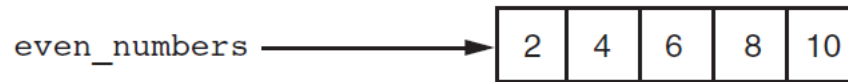
Figure 7-3 A list holding different types



```
info = ['Alicia', 27, 1550.87]
```


Introduction to Lists (cont'd.)

Figure 7-1 A list of integers



```
>>> even_numbers = [2, 4, 6, 8, 10] ◀
```



```
>>> Print(even_numbers) ◀
```



```
[2, 4, 6, 8, 10]
```

```
>>>
```

Introduction to Lists (cont'd.)

```
>>> numbers = list(range(5))  ␣
```



```
>>> Print(numbers)  ␣
```



```
[0, 1, 2, 3, 4]  
>>>
```

```
>>> numbers = list(range(1,10,2))  ␣
```



```
>>> Print(numbers)  ␣
```



```
[1, 3, 5, 7, 9]  
>>>
```

The Repetition Operator and Iterating over a List

- **Repetition operator**: makes multiple copies of a list and joins them together
 - The `*` symbol is a repetition operator when applied to a sequence and an integer
 - Sequence is left operand, number is right
 - General format: `list * n`

`numbers = [0] * 5`  `print(numbers)`  `[0, 0, 0, 0, 0]`

`numbers = [1,2,3] * 3`  `print(numbers)`  `[1, 2, 3, 1, 2, 3, 1, 2, 3]`

The Repetition Operator and Iterating over a List

- Repetition operator: makes multiple copies of a list and joins them together
- **You can iterate over a list using a `for` loop**
 - Format: `for x in list:`

```
numbers = [99, 100, 101, 102]
for n in numbers:
    print(n)
```



```
99
100
101
102
```

Indexing

- **Index**: a number specifying the position of an element in a list
 - Enables access to individual element in list
 - Index of first element in the list is 0, second element is 1, and nth element is n-1

my_list = [10, 20, 30, 40]

 `print(my_list[0], my_list[1], my_list[2], my_list[3])`  10, 20, 30, 40

`index = 0`
`while index < 4:`
 `print(my_list[index])` 
 `index += 1`
10
20
30
40

Indexing

- **Negative indexes identify positions relative to the end of the list**
 - The index -1 identifies the last element, -2 identifies the next to last element, etc.

my_list = [10, 20, 30, 40]



`print(my_list[-1], my_list[-2], my_list[-3], my_list[-4])` ↵ ➡ 40 30 20 10

Lists Are Mutable

- **Mutable sequence: the items in the sequence can be changed**
 - Lists are mutable, and so their elements can be changed
- **An expression such as**
`list[1] = new_value` **can be used to assign a new value to a list element**
 - Must use a valid index to prevent raising of an `IndexError` exception

Lists Are Mutable

1 numbers = [1, 2, 3, 4, 5]

2 print(numbers)

3 numbers[0] = 99

4 print(numbers)



The statement in line 2 will display
[1, 2, 3, 4, 5]



The statement in line 4 will display
[99, 2, 3, 4, 5]

Lists Are Mutable (Example)

Program 7-1

```
1 # The NUM_DAYS constant holds the number of
2 # days that we will gather sales data for.
3 NUM_DAYS = 5
4
5 def main():
6     # Create a list to hold the sales
7     # for each day.
8     sales = [0] * NUM_DAYS
9
10    # Create a variable to hold an index.
11    index = 0
12
13    print('Enter the sales for each day.')
14
15    # Get the sales for each day.
16    while index < NUM_DAYS:
17        print('Day #', index + 1, ': ', sep="", end="")
18        sales[index] = float(input())
19        index += 1
20
21    # Display the values entered.
22    print('Here are the values you entered:')
23    for value in sales:
24        print(value)
25
26 # Call the main function.
27 main()
```



Program Output (with input shown in bold)

Enter the sales for each day.

Day #1: **1000** Enter ↵

Day #2: **2000** Enter ↵

Day #3: **3000** Enter ↵

Day #4: **4000** Enter ↵

Day #5: **5000** Enter ↵

Here are the values you entered:

1000.0

2000.0

3000.0

4000.0

5000.0



sales_list.py



An IndexError exception

- **An IndexError exception is raised if an invalid index is used**

```
# This code will cause an IndexError exception.  
my_list = [10, 20, 30, 40]  
index = 0  
while index < 5:  
    print(my_list[index])  
    index += 1
```

The last time that this loop begins an iteration, the index variable will be assigned the value 4, which is an invalid index for the list. As a result, the statement that calls the print function will cause an **IndexError** exception to be raised.

The len function

- **len function**: returns the length of a sequence such as a list
 - Example: `size = len(my_list)`
 - Returns the number of elements in the list, so the index of last element is `len(list) - 1`
 - Can be used to prevent an `IndexError` exception when iterating over a list with a loop

```
my_list = [10, 20, 30, 40]
index = 0
while index < len(my_list):
    print(my_list[index])
    index += 1
```



```
10
20
30
40
```

The for Loop

```
my_list = ["a", "b", "c", "d", "e", "f"]  
print(my_list)
```

```
for var in my_list:  
    print(var)
```

['a', 'b', 'c', 'd', 'e', 'f']

a

b

c

d

e

f

The for Loop

```
my_list = ["a", "b", "c", "d", "e", "f"]  
print(my_list)
```

```
for i in range(len(my_list)):  
    print(my_list[i])
```

['a', 'b', 'c', 'd', 'e', 'f']

a

b

c

d

e

f

The enumerate Function

```
my_list = ["a", "b", "c", "d", "e", "f"]  
print(my_list)  
  
for i, var in enumerate(my_list):  
    print(i, var)
```

```
['a', 'b', 'c', 'd', 'e', 'f']  
0 a  
1 b  
2 c  
3 d  
4 e  
5 f
```

The enumerate Function

```
my_list = ["a", "b", "c", "d", "e", "f"]  
print(my_list)  
  
print(enumerate(my_list))  
  
print(type(enumerate(my_list)))  
  
print(list(enumerate(my_list)))
```

['a', 'b', 'c', 'd', 'e', 'f']

<enumerate object at 0x000001A42238D3A0>

<class 'enumerate'>

[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd'), (4, 'e'), (5, 'f')]

The range Function

```
my_list = range(0,100,10)
print(my_list)
print(type(my_list))

my_list = list(range(0,100,10))
print(my_list)
print(type(my_list))
```

range(0, 100, 10)

<class 'range'>

[0, 10, 20, 30, 40, 50, 60, 70, 80, 90]

<class 'list'>

Concatenating Lists

- **Concatenate: join two things together**
- **The + operator can be used to concatenate two lists**
 - Cannot concatenate a list with another data type, such as a number
- **The += augmented assignment operator can also be used to concatenate lists**

Concatenating Lists (Example)

```
list1 = [1, 2, 3, 4]  
list2 = [5, 6, 7, 8]  
list3 = list1 + list2
```

→ print(list3) → [1, 2, 3, 4, 5, 6, 7, 8]

```
girl_names = ['Joanne', 'Karen', 'Lori']  
boy_names = ['Chris', 'Jerry', 'Will']  
all_names = girl_names + boy_names
```



```
print(all_names)
```



```
['Joanne', 'Karen', 'Lori', 'Chris', 'Jerry', 'Will']
```



Topics

- Sequences
- Introduction to Lists
- **List Slicing**
- Finding Items in Lists with the in Operator
- List Methods and Useful Built-in Functions

List Slicing

- **Slice**: a span of items that are taken from a sequence
 - List slicing format: `list[start : end: step]`
 - Span is a list containing copies of elements from `start` up to, but not including, `end`
 - If `start` not specified, 0 is used for start index
 - If `end` not specified, `len(list)` is used for end index
 - If `step` not specified, 1 is used for the step.
- Slicing expressions can include a step value and negative indexes relative to end of list
- Can handle not existing indices, i.e. when `|start| > len(list)` or/and `|end| > len(list)`

`days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday']`

`mid_days = days[2:5]` → `print(mid_days)` → `['Tuesday', 'Wednesday', 'Thursday']`



`mid_days = days[88:99]` → `print(mid_days)` → `[]`

`days[ :: -1] ??`

Topics

- Sequences
- Introduction to Lists
- List Slicing
- **Finding Items in Lists with the in Operator**
- List Methods and Useful Built-in Functions

Finding Items in Lists with the `in` Operator

- You can use the `in` operator to determine whether an item is contained in a list
 - General format: `item in list`
 - Returns `True` if the item is in the list, or `False` if it is not in the list
- Similarly you can use the `not in` operator to determine whether an item is not in a list

Finding Items in Lists with the `in` Operator (Example)

Program 7-2 (`in_list.py`)

```
1 # This program demonstrates the in operator
2 # used with a list.
3
4 def main():
5     # Create a list of product numbers.
6     prod_nums = ['V475', 'F987', 'Q143', 'R688']
7
8     # Get a product number to search for.
9     search = input('Enter a product number: ')
10
11     # Determine whether the product number is in the list.
12     if search in prod_nums:
13         print(search, 'was found in the list.')
14     else:
15         print(search, 'was not found in the list.')
16
17 # Call the main function.
18 main()
```



Program Output (with input shown in bold)

Enter a product number: **Q143** ↵
Q143 was found in the list.

Program Output (with input shown in bold)

Enter a product number: **B000** ↵
B000 was not found in the list.



`in_list.py`



Topics

- Sequences
- Introduction to Lists
- List Slicing
- Finding Items in Lists with the in Operator
- **List Methods and Useful Built-in Functions**

List Methods and Useful Built-in Functions

- `append(item)` : used to add items to a list – *item* is appended to the end of the existing list
- `index(item)` : used to determine where an item is located in a list
 - Returns the index of the first element in the list containing *item*
 - Raises `ValueError` exception if *item* not in the list

List Methods and Useful Built-in Functions (cont'd.)

- `insert(index, item)`: used to insert *item* at position *index* in the list
- `sort()`: used to sort the elements of the list in **ascending order** (in ABC its according to the ASCII code so there is difference between upper and lower case letters). **Throw an exception if the list contains both strings and numeric values.**
- `remove(item)`: removes the first occurrence of *item* in the list
- `reverse()`: reverses the order of the elements in the list

Table 7-1 A few of the list methods

Method	Description
<code>append(<i>item</i>)</code>	Adds <i>item</i> to the end of the list.
<code>index(<i>item</i>)</code>	Returns the index of the first element whose value is equal to <i>item</i> . A <code>ValueError</code> exception is raised if <i>item</i> is not found in the list.
<code>insert(<i>index</i>, <i>item</i>)</code>	Inserts <i>item</i> into the list at the specified <i>index</i> . When an item is inserted into a list, the list is expanded in size to accommodate the new item. The item that was previously at the specified index, and all the items after it, are shifted by one position toward the end of the list. No exceptions will occur if you specify an invalid index. If you specify an index beyond the end of the list, the item will be added to the end of the list. If you use a negative index that specifies an invalid position, the item will be inserted at the beginning of the list.
<code>sort()</code>	Sorts the items in the list so they appear in ascending order (from the lowest value to the highest value).
<code>remove(<i>item</i>)</code>	Removes the first occurrence of <i>item</i> from the list. A <code>ValueError</code> exception is raised if <i>item</i> is not found in the list.
<code>reverse()</code>	Reverses the order of the items in the list.

List Methods and Useful Built-in Functions (cont'd.)

- **del statement**: removes an element from a specific index in a list
 - General format: `del list[i]`
Or: `del list[2:5]`
- **min and max functions**: built-in functions that returns the item that has the lowest or highest value in a sequence
 - The sequence is passed as an argument

```
num = [1, 3, 2, 8, 5, 10, 6]  
print('Maximum is:', max(num))
```

List Methods and Useful Built-in Functions (Example)

```
# This program demonstrates how the append
# method can be used to add items to a list.

def main():
    # First, create an empty list.
    name_list = []

    # Create a variable to control the loop.
    again = 'Y'

    # Add some names to the list.
    while again.upper() == 'Y':
        # Get a name from the user.
        name = input('Enter a name: ')

        # Append the name to the list.
        name_list.append(name)

        # Add another one?
        print('Do you want to add another name?')
        again = input('y = yes, anything else = no: ')
        print()

    # Display the names that were entered.
    print('Here are the names you entered.')

    for name in name_list:
        print(name)

# Call the main function.
main()
```



Program Output (with input shown in bold)

Enter a name: **Kathryn** ↵
Do you want to add another name?
y = yes, anything else = no: **y**

Enter a name: **Chris** ↵
Do you want to add another name?
y = yes, anything else = no: **y**

Enter a name: **Kenny** ↵
Do you want to add another name?
y = yes, anything else = no: **y**

Enter a name: **Renee** ↵
Do you want to add another name?
y = yes, anything else = no: **n**

Here are the names you entered.
Kathryn
Chris
Kenny
Renee



list_append.py

The index Method (Example)

Program 7-4 (index_list.py)

```
# This program demonstrates how to get the
# index of an item in a list and then replace
# that item with a new item.

def main():
    # Create a list with some items.
    food = ['Pizza', 'Burgers', 'Chips']

    # Display the list.
    print('Here are the items in the food list:')
    print(food)

    # Get the item to change.
    item = input('Which item should I change? ')

    try:
        # Get the item's index in the list.
        item_index = food.index(item)

        # Get the value to replace it with.
        new_item = input('Enter the new value: ')

        # Replace the old item with the new item.
        food[item_index] = new_item

        # Display the list.
        print('Here is the revised list:')
        print(food)
    except ValueError:
        print('That item was not found in the list.')

# Call the main function.
main()
```



Program Output (with input shown in bold)

Here are the items in the food list:

['Pizza', 'Burgers', 'Chips']

Which item should I change? **Burgers**

Enter the new value: **Pickles**

Here is the revised list:

['Pizza', 'Pickles', 'Chips']



index_list.py

The insert Method (Example)

Program 7-5 (insert_list.py)

This program demonstrates the insert method.

```
def main():  
    # Create a list with some names.  
    names = ['James', 'Kathryn', 'Bill']  
  
    # Display the list.  
    print('The list before the insert:')  
    print(names)  
  
    # Insert a new name at element 0.  
    names.insert(0, 'Joe')  
  
    # Display the list again.  
    print('The list after the insert:')  
    print(names)  
  
# Call the main function.  
main()
```



Program Output

The list before the insert:

['James', 'Kathryn', 'Bill']

The list after the insert:

['Joe', 'James', 'Kathryn', 'Bill']



insert_list.py

The sort Method (Example)

Example I:

```
my_list = [9, 1, 0, 2, 8, 6, 7, 4, 5, 3]
print('Original order:', my_list)
my_list.sort()
print('Sorted order:', my_list)
```



Original order: [9, 1, 0, 2, 8, 6, 7, 4, 5, 3]
Sorted order: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Example II:

```
my_list = ['beta', 'alpha', 'delta', 'gamma']
print('Original order:', my_list)
my_list.sort()
print('Sorted order:', my_list)
```



Original order: ['beta', 'alpha', 'delta', 'gamma']
Sorted order: ['alpha', 'beta', 'delta', 'gamma']

The reverse Method (Example)

Example I:

```
my_list = [9, 1, 0, 2]
print('Original order:', my_list)
my_list.reverse()
print('Reversed order:', my_list)
```



Original order: [9, 1, 0, 2]
Reversed order: [2, 0, 1, 9]

Example II:

```
my_list = ['beta', 'alpha', 'delta', 'gamma']
print('Original order:', my_list)
my_list.reverse()
print('Reversed order:', my_list)
```



Original order: ['beta', 'alpha', 'delta', 'gamma']
Reversed order: ['gamma', 'delta', 'alpha', 'beta']

The remove Method (Example)

Program 7-6 (remove_item.py)

```
# This program demonstrates how to use the remove
# method to remove an item from a list.

def main():
    # Create a list with some items.
    food = ['Pizza', 'Burgers', 'Chips']

    # Display the list.
    print('Here are the items in the food list:')
    print(food)

    # Get the item to change.
    item = input('Which item should I remove? ')

    try:
        # Remove the item.
        food.remove(item)

        # Display the list.
        print('Here is the revised list:')
        print(food)

    except ValueError:
        print('That item was not found in the list.')

# Call the main function.
main()
```



Program Output

Here are the items in the food list:

['Pizza', 'Burgers', 'Chips']

Which item should I remove? **Burgers**

Here is the revised list:

['Pizza', 'Chips']



remove_item.py

The del Statement (Example)

```
my_list = [1, 2, 3, 4, 5]  
print('Before deletion:', my_list)  
del my_list[2]  
print('After deletion:', my_list)
```



This code will display the following:



```
Before deletion: [1, 2, 3, 4, 5]  
After deletion: [1, 2, 4, 5]
```

The min and max Functions (Example)

```
my_list = [5, 4, 3, 2, 50, 40, 30]  
print('The lowest value is',  
min(my_list))
```



This code will display the following:



The lowest value is 2

```
my_list = [5, 4, 3, 2, 50, 40, 30]  
print('The highest value is', max(my_list))
```



This code will display the following:



The highest value is 50



Topics (cont'd.)

- **Copying Lists**
- Processing Lists
- Two-Dimensional Lists
- Tuples
- Plotting List Data with the `matplotlib` Package

Copying Lists

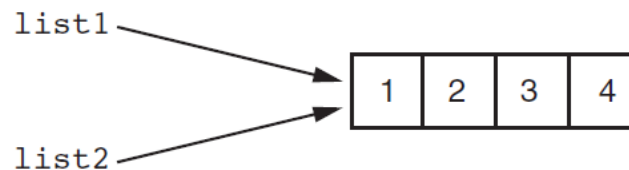
- **To make a copy of a list you must copy each element of the list**
 - Two methods to do this:
 1. Creating a new empty list and using a `for` loop to add a copy of each element from the original list to the new list
 2. Creating a new empty list and concatenating the old list to the new empty list

Copying Lists (cont'd.)

```
# Create a list.  
list1 = [1, 2, 3, 4]  
# Assign the list to the list2 variable.  
list2 = list1
```

After this code executes, both variables `list1` and `list2` will reference the same list in memory. This is shown in Figure 7-4

Figure 7-4 `list1` and `list2` reference the same list

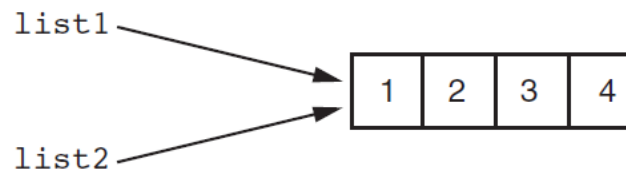


Copying Lists (cont'd.)

```
1 >>> list1 = [1, 2, 3, 4] ↵
2 >>> list2 = list1 ↵
3 >>> print(list1) ↵
4 [1, 2, 3, 4]
5 >>> print(list2) ↵
6 [1, 2, 3, 4]
7 >>> list1[0] = 99 ↵
8 >>> print(list1) ↵
9 [99, 2, 3, 4]
10 >>> print(list2) ↵
11 [99, 2, 3, 4]
12 >>>
```

In this interactive session, the list1 and list2 variables reference the same list in memory

Figure 7-4 list1 and list2 reference the same list



Copying Lists (1)

```
# Create a list with values.
```

```
list1 = [1, 2, 3, 4]
```

```
# Create an empty list.
```

```
list2 = [ ]
```

```
# Copy the elements of list1 to list2.
```

```
for item in list1:
```

```
    list2.append(item)
```

Suppose you wish to make a copy of the list, so list1 and list2 reference two separate but identical lists.

One way to do this is with a loop that copies each element of the list. Here is an example:

Copying Lists (2)

Create a list with values.

```
list1 = [1, 2, 3, 4]
```

Create a copy of list1.

```
list2 = [ ] + list1
```

A simpler and more elegant way to accomplish the same task is to use the concatenation operator, as shown here:



Topics

- Copying Lists
- **Processing Lists**
- Two-Dimensional Lists
- Tuples
- Plotting List Data with the `matplotlib`
Package

Processing Lists

- **List elements can be used in calculations**
- **To calculate total of numeric values in a list use loop with accumulator variable**
- **To average numeric values in a list:**
 - Calculate total of the values
 - Divide total of the values by `len(list)`
- **List can be passed as an argument to a function**

Processing Lists (Example1)

Program 7-8 (total_list.py)

```
1 # This program calculates the total of the values
2 # in a list.
3
4 def main():
5     # Create a list.
6     numbers = [2, 4, 6, 8, 10]
7
8     # Create a variable to use as an accumulator.
9     total = 0
10
11     # Calculate the total of the list elements.
12     for value in numbers:
13         total += value
14
15     # Display the total of the list elements.
16     print('The total of the elements is', total)
17
18 # Call the main function.
19 main()
```



Program Output:

The total of the elements is 30



total_list.py



Processing Lists (Example2)

Program 7-9 (average_list.py)

```
1 # This program calculates the average of the values
2 # in a list.
3
4 def main():
5     # Create a list.
6     scores = [2.5, 7.3, 6.5, 4.0, 5.2]
7
8     # Create a variable to use as an accumulator.
9     total = 0.0
10
11     # Calculate the total of the list elements.
12     for value in scores:
13         total += value
14
15     # Calculate the average of the elements.
16     average = total / len(scores)
17
18     # Display the total of the list elements.
19     print('The average of the elements is', average)
20
21 # Call the main function.
22 main()
```



The average of the elements is 5.3



average_list.py



Processing Lists (cont'd.)

- **A function can return a reference to a list**
- To save the contents of a list to a file:
 - Use the file object's `writelines` method
 - Does not automatically write `\n` at the end of each item
 - Use a for loop to write each element and `\n`
- To read data from a file use the file object's `readlines` method

Processing Lists (cont'd.)

```
1 # This program uses a function to calculate the
2 # total of the values in a list.
3
4 def main():
5     # Create a list.
6     numbers = [2, 4, 6, 8, 10]
7
8     # Display the total of the list elements.
9     print('The total is', get_total(numbers))
10
11 # The get_total function accepts a list as an
12 # argument returns the total of the values in
13 # the list.
14 def get_total(value_list):
15     # Create a variable to use as an accumulator.
16     total = 0
17
18     # Calculate the total of the list elements.
19     for num in value_list:
20         total += num
21
22     # Return the total.
23     return total
24
25 # Call the main function.
26 main()
```



Program Output
The total is 30

Passing a List as an
Argument to a Function

Function returns a reference to a list #1

Program 7-10
(total_function.py)



total_function.py

Processing Lists (cont'd.)

Returning a List from a Function

```
1 # This program uses a function to create a list.
2 # The function returns a reference to the list.
4 def main():
5     # Get a list with values stored in it.
6     numbers = get_values()
7
8     # Display the values in the list.
9     print('The numbers in the list are:')
10    print(numbers)
11
12    # The get_values func. gets a series of numbers
13    # from the user and stores them in a list. The
14    # function returns a reference to the list.
15    def get_values():
16        # Create an empty list.
17        values = [ ]
18
19        # Create a variable to control the loop.
20        again = 'y'
21
22        # Get values from the user and add them to
23        # the list.
24        while again == 'y':
25            # Get a number and add it to the list.
26            num = int(input('Enter a number: '))
27            values.append(num)
28
29            # Want to do this again?
30            print('Do you want to add another number?')
31            again = input('y = yes, anything else = no: ')
32            print()
33
34        # Return the list.
35        return values
36
37    # Call the main function.
38    main()
```

Program 7-11
(return_list.py)



return_list.py

Processing Lists (cont'd.)

Returning a List from a Function

Program Output (with input shown in bold)

Enter a number: **1** ↵
Do you want to add another number?
y = yes, anything else = no: **y** ↵
Enter a number: **2** ↵
Do you want to add another number?
y = yes, anything else = no: **y** ↵
Enter a number: **3** ↵
Do you want to add another number?
y = yes, anything else = no: **y** ↵
Enter a number: **4** ↵
Do you want to add another number?
y = yes, anything else = no: **y** ↵
Enter a number: **5** ↵
Do you want to add another number?
y = yes, anything else = no: **n** ↵
The numbers in the list are:
[1, 2, 3, 4, 5]

Program 7-11
(return_list.py)



return_list.py

Topics

- Copying Lists
- Processing Lists
- **Two-Dimensional Lists**
- Tuples
- Plotting List Data with the `matplotlib`
Package

Two-Dimensional Lists

- **Two-dimensional list: a list that contains other lists as its elements**
 - Also known as nested list
 - Common to think of two-dimensional lists as having rows and columns
 - Useful for working with multiple sets of data
- **To process data in a two-dimensional list need to use two indexes**
- **Typically use nested loops to process**

Two-Dimensional Lists

- **Example**

```
1 >>> students = [['Joe', 'Kim'], ['Sam', 'Sue'], ['Kelly', 'Chris']] ↵
2 >>> print(students) ↵
3 [ ['Joe', 'Kim'], ['Sam', 'Sue'], ['Kelly', 'Chris'] ]
4 >>> print(students[0]) ↵
5 ['Joe', 'Kim']
6 >>> print(students[1]) ↵
7 ['Sam', 'Sue']
8 >>> print(students[2]) ↵
9 ['Kelly', 'Chris']
10 >>>
```

Line 1 creates a list and assigns it to the students variable. The list has three elements, and each element is also a list. The element at students[0] is ['Joe', 'Kim']

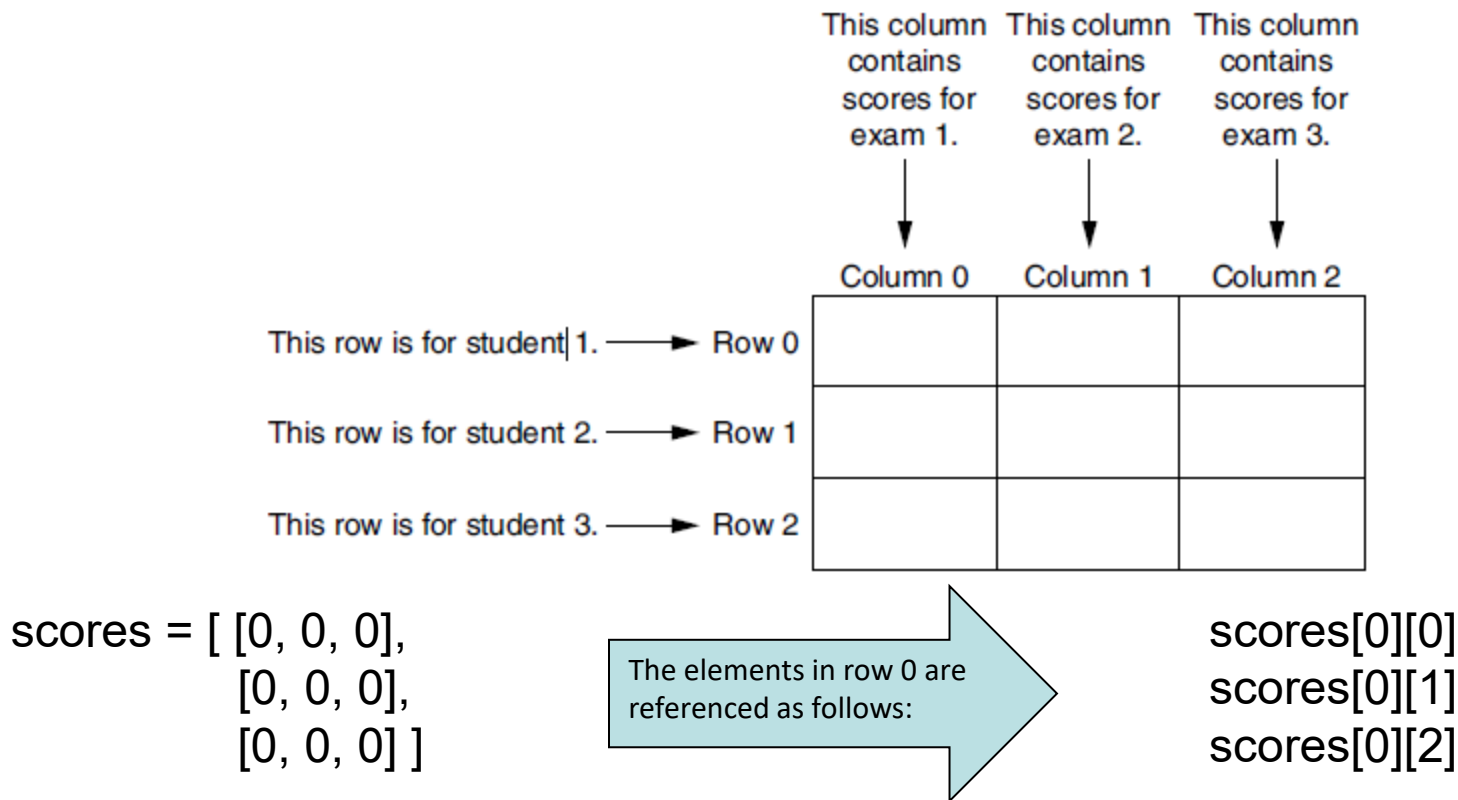
Two-Dimensional Lists (cont'd.)

Figure 7-5 A two-dimensional list

	Column 0	Column 1
Row 0	'Joe '	'Kim '
Row 1	'Sam '	'Sue '
Row 2	'Kelly '	'Chris '

Two-Dimensional Lists (cont'd.)

● **Figure 7-6** Two-dimensional list with three rows and three columns



Two-Dimensional Lists (cont'd.)

Figure 7-7 Subscripts for each element of the `scores` list

	Column 0	Column 1	Column 2
Row 0	<code>scores[0][0]</code>	<code>scores[0][1]</code>	<code>scores[0][2]</code>
Row 1	<code>scores[1][0]</code>	<code>scores[1][1]</code>	<code>scores[1][2]</code>
Row 2	<code>scores[2][0]</code>	<code>scores[2][1]</code>	<code>scores[2][2]</code>

Subscripts for each element of the `scores` list

Two-Dimensional Lists (Example)

```
my_list = [[1,2,3,"a"],["x","y","z"],[11.1,12.5,3,4,5]]  
print(my_list)  
  
for i in range(len(my_list)):  
    print(my_list[i])
```

```
[[1, 2, 3, 'a'], ['x', 'y', 'z'], [11.1, 12.5, 3, 4, 5]]  
[1, 2, 3, 'a']  
['x', 'y', 'z']  
[11.1, 12.5, 3, 4, 5]
```

Two-Dimensional Lists (Example)

```
my_list = [[1,2,3,"a"],["x","y","z"],[11.1,12.5,3,4,5]]  
print(my_list)
```

```
for i in range(len(my_list)):  
    print(my_list[i])  
    for j in range(len(my_list[i])):  
        print(my_list[i][j])
```

```
[[1, 2, 3, 'a'], ['x', 'y', 'z'], [11.1, 12.5, 3, 4, 5]]  
[1, 2, 3, 'a']  
1  
2  
3  
a  
['x', 'y', 'z']  
x  
y  
z  
[11.1, 12.5, 3, 4, 5]  
11.1  
12.5  
3  
4  
5
```

Two-Dimensional Lists (Example)

```
my_list = [[1,2,3,"a"],["x","y","z"],[11.1,12.5,3,4,5]]  
  
for row in my_list:  
    print(row)
```

```
[1, 2, 3, 'a']  
['x', 'y', 'z']  
[11.1, 12.5, 3, 4, 5]
```

Two-Dimensional Lists (Example)

```
my_list = [[1,2,3,"a"],["x","y","z"],[11.1,12.5,3,4,5]]
```

```
for row in my_list:
```

```
    print(row)
```

```
    for col in row:
```

```
        print(col)
```

```
[1, 2, 3, 'a']
```

```
1
```

```
2
```

```
3
```

```
a
```

```
['x', 'y', 'z']
```

```
x
```

```
y
```

```
z
```

```
[11.1, 12.5, 3, 4, 5]
```

```
11.1
```

```
12.5
```

```
3
```

```
4
```

```
5
```



Topics

- Copying Lists
- Processing Lists
- Two-Dimensional Lists
- **Tuples**
- Plotting List Data with the `matplotlib`
Package

Tuples

- **Tuple: an immutable sequence**
 - Very similar to a list
 - Once it is created it cannot be changed
 - Format: `tuple_name = (item1, item2)`
 - Tuples support operations as lists
 - Subscript indexing for retrieving elements
 - Methods such as **`index`**
 - Built in functions such as **`len`, `min`, `max`**
 - Slicing expressions
 - The **`in`** operator
 - The **`+`** and **`*`** operators

Tuples (Examples)

```
>>> my_tuple = (1, 2, 3, 4, 5) ↵
```

```
>>> print(my_tuple) ↵
```

```
(1, 2, 3, 4, 5)
```

```
>>>
```

```
>>> # Loop iteration over the elements in a tuple
```

```
>>> names = ('Holly', 'Warren', 'Ashley') ↵
```

```
>>> for n in names: ↵
```

```
    print(n) ↵
```

```
Holly
```

```
Warren
```

```
Ashley
```

```
>>>
```

Tuples (Examples)

```
>>> # Like lists, tuples support indexing:  
>>> names = ('Holly', 'Warren', 'Ashley')  
>>> for i in range(len(names)):  
    print(names[i])  
  
Holly  
Warren  
Ashley  
  
>>>
```


Tuples (cont'd.)

- **Tuples do not support the methods:**
 - `append`
 - `remove`
 - `insert`
 - `reverse`
 - `sort`

NOTE: If you want to create a tuple with just one element, you must write a trailing comma after the element's value, as shown here:

```
my_tuple = (1,)    # Creates a tuple with one element.
```

If you omit the comma, you will not create a tuple. For example, the following statement simply assigns the integer value 1 to the value variable:

```
value = (1)        # Creates an integer.
```

Tuples (cont'd.)

- **Advantages for using tuples over lists:**
 - Processing tuples is faster than processing lists
 - Tuples are safe
 - Some operations in Python require use of tuples

Tuples (cont'd.)

- **list() function: converts tuple to list**

```
>>> my_tuple = (1, 2, 3, 4, 5) ↵
```

```
>>> print(my_tuple) ↵
```

```
(1, 2, 3, 4, 5)
```

```
>>> my_tuple=list(my_tuple) ↵
```

```
>>> print(my_tuple) ↵
```

```
[1, 2, 3, 4, 5]
```

Tuples (cont'd.)

- **tuple() function: converts list to tuple**

```
>>> my_tuple = [1, 2, 3, 4, 5] ↵
```

```
>>> print(my_tuple) ↵
```

```
[1, 2, 3, 4, 5]
```

```
>>> my_tuple=tuple(my_tuple)
```

```
>>> print(my_tuple) ↵
```

```
(1, 2, 3, 4, 5)
```

Tuples (Example)

```
1 >>> number_tuple = (1, 2, 3) ↵  
2 >>> number_list = list(number_tuple) ↵  
3 >>> print(number_list) ↵  
4 [1, 2, 3]  
5 >>> str_list = ['one', 'two', 'three'] ↵  
6 >>> str_tuple = tuple(str_list) ↵  
7 >>> print(str_tuple) ↵  
8 ('one', 'two', 'three')  
9 >>>
```



Tuples - Motivation

להוסיף השמה של שני משתנים והחלפה שלהם

```
(x , y, z) = (1, 2, 3)  
print(x, y, z)
```

בכל מקום שפייתון יודע שמדובר בטאפלים ניתן להוריד את
הסוגריים

```
x , y, z = 1, 2, 3
```



Tuples - Motivation

השמה של שני משתנים והחלפה שלהם

```
x=4
```

```
y=5
```

```
print(x,y)
```

```
(x , y) = (y, x)
```

```
print(x,y)
```

ואפשר גם בלי סוגריים

```
x , y = y , x
```



Tuples - Motivation

כאשר פונקציות מחזירות הרבה ערכים, זה בעצם רשומה אחת שמכילה אוסף ערכים.

```
def func(x,y):  
    return (x+y, 2*x)
```

```
a, b = func(2, 3)  
c = func(1,7)
```

```
def func(x,y):  
    return x+y, 2*x  
  
a, b = func(2, 3)  
c = func(1,7)
```



Summary

- **This chapter covered:**
 - Lists, including:
 - Repetition and concatenation operators
 - Indexing
 - Techniques for processing lists
 - Slicing and copying lists
 - List methods and built-in functions for lists
 - Two-dimensional lists
 - Tuples, including:
 - Immutability
 - Difference from and advantages over lists